Quick start

This guide takes you from a fresh checkout to a document that runs, is saved, and can be checked for reproducibility. It covers what the program does today. Where a feature exists only as a library call, the guide says so.

1. Install the toolchain

Install Rust with rustup. The repository's `rust-toolchain.toml` selects the right channel, so `cargo` picks it up automatically.

The first build downloads the TPT foundation crates from GitHub and crates.io, so it needs network access. After that the program runs offline.

```
cargo build --workspace
cargo test --workspace
```

2. Look at the example project

The repository includes a demo project with two documents, an ODE solve, a units cell, sampling and a linear solve.

```
cargo run -p research-app --example demo_project -- demo-project
```

This creates `demo-project/`, runs every document, and prints each cell's outputs.

3. Open a project in the window

```
cargo run -p research-app -- demo-project
```

The window opens the project in that directory. If the directory does not exist, it is created. Only the directory on the command line is used, so there is no open or create dialog yet.

The window has three areas:

- **Left:** the document list, the run list, and the settings.
- **Middle:** comparison results, when you have made one.
- **Right:** the open document.

4. Write and run a document

1. Click **New document**. It is titled `Untitled N` and opens on the right.
2. Click **Add code cell**, then type in the box: for example `let k = 2.5; k * 4.0`.
3. Click **Run**. The status line says how many cells ran and how many were kept. Within one session, a cell that has not changed since its last run is kept. After you reopen a project, the first **Run** reruns every cell. The variables from earlier cells are not saved between sessions, so a later cell could not read them otherwise. The reruns are deterministic, so the results match.
4. Click **Save**. The document and its outputs are written to the project.

Code cells are written in Rhai. Use decimal literals for floats (`2.0`, not `2`) in the math functions. Two cell functions are worth knowing:

- `show_table(columns, rows)` shows a table.
- `show_plot(svg)` records a plot. The argument must be SVG text that starts with `<svg`. The window shows a plot as a size line, not as an image.

Add text cell adds a markdown cell for notes. Text cells are shown as plain text.

5. Compare runs

Each cell that runs is recorded as a run.

1. Tick two or more runs in the run list.
2. Click **Compare N selected**. The comparison shows which inputs differ and each run's results side by side.
3. Changing the ticks clears the comparison. **Clear comparison** unticks everything.

6. Check reproducibility

A check reruns the open document from scratch, with the same seed, and compares each code cell's new outputs with the stored ones.

1. In **Settings** in the left panel, click **Agents: off** so it reads **Agents: on**. Agents are off by default, and the check refuses to run while they are off.
2. Click **Check reproducibility**.

The status line reports either that the cells match, or which cells differ. Each check uses a fresh agent that can only rerun that one document, and the agent's actions are kept in the session.

The check uses the open document as it is on screen, so save first if you want the check to reflect your latest edits.

7. Packages, notebooks and other features

These features exist in the libraries but have no window or command yet:

- **Packages.** The `research-package` crate packs a project into one signed file, optionally encrypted, and unpacks it elsewhere with every file hash-checked. Packing has no size limit of its own. Import refuses packages larger than 2 GiB in total. See PACKAGE_FORMAT.md.
- **Jupyter notebooks.** The `research-doc` crate imports and exports `.ipynb` files. Text, errors, SVG plots and tables are kept (tables come back as text). Other rich outputs, such as images and HTML, are dropped on import with a warning. See DOCUMENT_FORMAT.md.

Where things are

- `demo-project/` (or your project directory): the project database, runs, results, documents and blobs. Do not edit these files by hand while the project is open.
- `docs/ARCHITECTURE.md`: how the crates fit together.
- `TODO.md`: what is done and what is open.

Troubleshooting

- **Build fails with a network error:** the first build needs GitHub and crates.io. Check your connection or proxy, then build again.
- **The window says "agents are off":** turn agents on in Settings, then check again.
- **A cell says `UnsupportedLanguage`:** the cell is not Rhai. Only Rhai cells run in this build.
- **A cell times out or hits a limit:** a cell has a time, memory and operation limit. Simplify the cell or split it into smaller cells.